

제 8 주:

스택: 수식계산



강 지 훈

jhkang@cnu.ac.kr

충남대학교 컴퓨터공학과



[제 8 주 실습]

스택 : 수식계산

□ 실습 목표

■ 이론적 관점

- 스택의 개념
- 수식의 표현 방법: infix, postfix
- Infix 수식을 postfix 수식으로 변환하는 방법
- Postfix 수식 계산 방법

■ 구현적 관점

- 스택을 이용하여 infix 수식을 postfix 수식으로 변환
- 스택을 이용하여 postfix 수식을 계산

과제에서 해결할 문제



□ [프로그램 O8]

- 키보드에서 한번에 하나의 문자열을 입력 받는다.
 - 문자열은 infix 수식을 나타낸다.
 - 수식 표현에 오류는 없다고 가정한다.
- 입력의 종료 조건
 - '!'키를 치면 프로그램을 종료한다.
- 매번 하나의 수식을 입력 받을 때마다 다음의 일을 한다.
 - Infix 수식을 postfix 형태로 변환한다.
 - Postfix 수식을 계산한다.

□ 이 과제에서 필요한 객체는?

- Class "AppController"

- Class "AppView"

- Model

 - Class "Calculate"

 - Class "ArrayList<T>" implements "Stack<T>"

 - Interface "Stack<T>"

□ 수식 계산 단계별 출력

■ 제1단계: 입력

- 키보드로부터 infix 수식을 입력 받는다.
 - ◆ 연산자: '+', '-', '*', '/', '%', '^' (정수연산만)
 - ◆ 연산값: '0' ~ '9'의 한자리 숫자 문자

■ 제2단계: Infix 수식을 Postfix 수식으로 변환

- 맨 처음 empty stack은 출력하지 않는다.
- 연산자를 만날 때마다, 그 처리 후의 스택 내용을 출력한다.
- 최종 변환된 수식을 출력한다.

■ 제3단계: Postfix 수식의 계산

- 맨 처음 empty stack은 출력하지 않는다.
- 스택이 변할 때마다 그 내용을 출력한다.
 - ◆ 연산값을 만나서 스택에 push 한 직후에
 - ◆ 연산자를 만나 계산이 이루어진 직후에
- 최종 계산 값을 출력한다.

□ 출력의 예

```
:: 프로그램을 시작합니다. ::
```

```
[수식 입력을 시작합니다.]
```

```
> 수식을 입력하시오: 3+5
```

```
[Infix를 Postfix로]
```

```
OStack : +
```

```
[Postfix] 35+
```

```
VStack : 3.0
```

```
VStack : 3.0 5.0
```

```
VStack : 8.0
```

```
[최종값]8.0
```

```
> 수식을 입력하시오: (9/(6-3))
```

```
[Infix를 Postfix로]
```

```
OStack : (
```

```
OStack : ( /
```

```
OStack : ( / (
```

```
OStack : ( / ( -
```

```
OStack : ( /
```

```
OStack :
```

```
[Postfix] 963-/
```

```
VStack : 9.0
```

```
VStack : 9.0 6.0
```

```
VStack : 9.0 6.0 3.0
```

```
VStack : 9.0 3.0
```

```
VStack : 3.0
```

```
[최종값]3.0
```

```
> 수식을 입력하시오: 9/6-3
```

```
[Infix를 Postfix로]
```

```
OStack : /
```

```
OStack : -
```

```
[Postfix] 96/3-
```

```
VStack : 9.0
```

```
VStack : 9.0 6.0
```

```
VStack : 1.5
```

```
VStack : 1.5 3.0
```

```
VStack : -1.5
```

```
[최종값]-1.5
```

```
> 수식을 입력하시오: (8-6)*(2+5*(7-4))
```

```
[Infix를 Postfix로]
```

```
OStack : (
```

```
OStack : ( -
```

```
OStack :
```

```
OStack : *
```

```
OStack : * (
```

```
OStack : * ( +
```

```
OStack : * ( + *
```

```
OStack : * ( + * (
```

```
OStack : * ( + * ( -
```

```
OStack : * ( + *
```

```
OStack : *
```

```
[Postfix] 86-2574-*+*
```

```
VStack : 8.0
```

```
VStack : 8.0 6.0
```

```
VStack : 2.0
```

```
VStack : 2.0 2.0
```

```
VStack : 2.0 2.0 5.0
```

```
VStack : 2.0 2.0 5.0 7.0
```

```
VStack : 2.0 2.0 5.0 7.0 4.0
```

```
VStack : 2.0 2.0 5.0 3.0
```

```
VStack : 2.0 2.0 15.0
```

```
VStack : 2.0 17.0
```

```
VStack : 34.0
```

```
[최종값]34.0
```

```
> 수식을 입력하시오: !
```

```
[수식 입력을 종료합니다.]
```

```
:: 프로그램을 종료합니다. ::
```

main()

□ main()을 위한 class

```
public class DS1_08_학번_이름 {  
    public static void main (String[] args)  
    {  
        ApplicationController appController = new ApplicationController() ;  
        // ApplicationController 가 실질적인 main class 이다.  
        appController.run() ;  
        // 여기 main()에서는 앱 실행이 시작되도록 해주는 일이 전부이다.  
    }  
}
```

Class "AppController"



□ Class "AppController" 의 구현 [1]

```
public class AppController {  
    // 비공개 변수들  
    private AppView _appView ;  
    private Calculate _calculate ;  
  
    // 생성자  
    public AppController()  
    {  
        this._appView = new AppView() ;  
        this._calculate = new Calculate() ;  
    }  
  
    // 비공개함수의 구현  
    .....  
  
    // 공개함수의 구현  
    .....  
} // End of class "AppController"
```

□ Class "AppController" 의 구현 [2] 수식 계산

```
public class AppController {  
    public void evalExpression()  
        // 주어진 infix 수식을 계산한다.
```

```
    public void evalExpression() {  
        double finalValue;  
        this.showMessage(MessageID.Notice_InfixToPostfix);  
  
        if (this._calculate.infixToPostfix()) {  
            this._appView.outputPostfix(this._calculate.postfix());  
            finalValue = this._calculate.evalPostfix();  
            this._appView.outputResult(finalValue);  
        } else  
            this.showMessage(MessageID.Error_Input);  
    }
```

□ Class "AppController" 의 구현: run()

```
public class AppController {
```

```
.....
```

```
public void run()
{
```

```
    // 프로그램의 실질적인 실행을 위한 함수. 실질적인 main() 함수.
```

```
    this.showMessage(MessageID.Notice_StartProgram);
```

```
    this.showMessage(MessageID.Notice_StartMenu);
```

```
    String input = this._appView.inputExpression();
```

```
    while(input.charAt(0) != '!') {
```

```
        this._calculate.setInfix(input);
```

```
        this.evalExpression();
```

```
        input = this._appView.inputExpression();
```

```
    }
```

```
    this.showMessage(MessageID.Notice_EndMenu);
```

```
    this.showMessage(MessageID.Notice_EndProgram);
```

```
}
```

Class "AppView"



□ AppView: 인스턴스 변수, 생성자

```
public class AppView {  
    // 비공개 상수/변수들  
    private Scanner _scanner ;  
  
    // 생성자  
    public AppView ()  
    {  
        this._scanner = new Scanner (System.in) ;  
    }  
  
    // 공개함수의 구현  
    .....  
}
```

□ Class "AppView" 의 공개함수

■ 입력을 위한 공개 함수

- public String inputString ()
- public String inputExpression ()
 - ◆ 수식을 입력 받는 함수

```
public String inputExpression()  
{  
    // 수식을 입력 받는 공개함수  
    System.out.print("> 수식을 입력하시오:");  
    return this.inputString();  
}
```

□ Class "AppView" 의 공개함수

■ 출력을 위한 공개 함수

- public void outputResult (double aValue)
 - ◆ 최종 값을 출력
- public void outputPostfix (String aPostfix)
 - ◆ Postfix를 출력
- public void outputMessage (String aMessage)

Enum "MessageID"

Enum "MessageID"

```
public enum MessageID {  
  
    // Message IDs for Notices:  
    Notice_StartProgram,  
    Notice_EndProgram,  
    Notice_StartMenu,  
    Notice_EndMenu,  
    Notice_InfixToPostfix,  
  
    // Error  
    Error_Input,  
  
} //End of enum "MessageID"
```

스택을 구현하는 Class "ArrayList"



□ Class “ArrayList<T>”

- [실습-07] 에 사용한 ArrayList<T> 를 그대로 복사하여 수정 없이 그대로 사용한다.

Class "Calculate"



□ Calculate: 생성자, 공개함수의 사용법

■ 생성자:

- `public Calculate ()`

■ 공개함수:

- `public void setInfix (String newInfix)`
 - ◆ 주어진 newInfix 를 Calculate 객체에게 보내어 보관시킨다.
- `public String infix ()`
 - ◆ Calculate 객체가 가지고 있는 infix 를 얻는다.
- `public String postfix ()`
 - ◆ Calculate 객체가 가지고 있는 postfix를 얻는다.
 - ◆ `infixToPostfix()` 를 실행한 후에 얻을 수 있다.
- `public boolean infixToPostfix ()`
 - ◆ Infix를 postfix로 변환한다. 변환된 수식을 보관한다.
 - ◆ 변환을 실행한 후의 성공 여부를 true/false 로 알려준다.
- `public double evalPostfix ()`
 - ◆ 현재 가지고 있는 postfix 수식을 계산하게 하여 그 결과를 얻는다.

□ Calculate: 인스턴스 변수

```
public class Calculate {
```

```
    private static final char END_OF_TOKEN = '$';
```

```
    private ArrayList<Character> _oStack ;
```

```
    private ArrayList<Double> _vStack ;
```

```
    private char[] _infix ;
```

```
    private char[] _postfix ;
```

□ Calculate: 생성자, 공개함수의 구현

■ 생성자:

- public Calculate()

- ◆ 사용 할 모든 private 멤버 변수를 초기화

■ 공개함수

- public void setInfix (String newInfix)

- ◆ 전달 받은 String 자료형의 newInfix 를, char[] 자료형인 this._infix[] 에 저장.
- ◆ String.toCharArray() 를 사용하여 자료형 변환

- public String infix()

- ◆ this._infix[] 를 String 으로 변환하여 돌려준다.
- ◆ String.valueOf() 를 사용하여 자료형 변환

- public String postfix()

- ◆ 현재 가지고 있는 postfix 를 돌려준다.
- ◆ String.valueOf() 를 이용 (char[] 를 String 으로 자료형 변환)

□ Calculate: 비공개 함수

■ private boolean isDigit (char aToken)

- 주어진 aToken 이 숫자인지 아닌지 확인하여 숫자일 경우 true 를 돌려준다.
 - ◆ 주어진 aToken 의 값이 '0' 과 '9' 이내의 값이면 숫자이다.
- 아닐 경우 false 를 돌려준다.

■ private int inComingPrecedence (char aToken)

- 현재 스택에 들어가야 하는 연산자 aToken 의 우선 순위 값을 돌려준다.

■ private int inStackPrecedence (char aToken)

- 스택 안에 존재 하는 동안의 연산자 aToken 의 우선 순위 값을 돌려준다.

□ Calculate: 예외적인 출력 [1]

■ 디버깅 목적의 예외적인 출력을 담당하는 함수:

● private void showOStackAll ()

- ◆ 디버깅 목적으로, 스택의 상태를 확인 하기 위하여 사용하는 함수
- ◆ Operation 스택 (this._oStack) 에 쌓여 있는 모든 원소들을 출력

● private void showVStackAll ()

- ◆ 디버깅 목적으로, 스택의 상태를 확인 하기 위하여 사용하는 함수
- ◆ Value 스택 (this._vStack) 에 쌓여 있는 모든 원소들을 출력

■ 실제 Calculate 객체는 계산이 주 목적으로, MVC 설계에서 Model 에 해당하는 객체이다. 그렇기 때문에 출력을 하는 부분이 존재 해서는 안 된다.

그렇지만 이번 과제에서는 실제 프로그램이 스택을 정상적으로 사용하고 있는지를 확인하려고 한다. 이러한 디버깅 목적으로 프로그램이 실행되는 동안 Stack이 변하는 모습을 보여주도록 임시로 추가 하여 사용한다.

□ Calculate: 예외적인 출력 [2]

■ private void showOStackAll ()

```
public void showOStackAll()
{
    // 임시함수. Operation 스택 내부에 있는 모든 값을 출력
    System.out.print("OStack: ");
    for(int index = 0; index < this._oStack.size(); index++) {
        System.out.print(this._oStack.elementAt(index) + " ");
    }
    System.out.println();
}
```

■ private void showVStackAll ()

- Value 스택 내부의 원소들을 모두 출력
- "showOStackAll ()" 을 참고하여 작성

Infix 를 Postfix 로

Member Functions의 구현

Calculate 의 Public Member function의 구현

```
public boolean infixToPostFix() {
    //infix를 postfix로 변경. 연산자의 우선순위에 유의
    int i = 0; // string pointer for infix
    int p = 0; //string pointer for postfix
    char currentToken, poppedToken, topToken;

    this._oStack = new ArrayList<Character>(this._infix.length);
    this._postfix = new char[this._infix.length+1];

    while(i < this._infix.length) {
        currentToken = this._infix[i++];
        if(this.isDigit(currentToken)) {
            // operand : output to postfix expression
            this._postfix[p++] = currentToken;
        }
        else { // currentToken 은 연산자
            if(currentToken == ')') { //currentToken이 괄호 연산자일때
                if(!this._oStack.isEmpty()) {
                    poppedToken = (char)this._oStack.pop();
                }
            }
            else
                return false;

            while(poppedToken != '(') {
                this._postfix[p++] = poppedToken;
                if(!this._oStack.isEmpty()) {
                    poppedToken = (char)this._oStack.pop();
                }
            }
        }
    }
}
```

함수의 알고리즘을 이해하고, 실제 입력 값을 예로 들어 소스에 대한 설명을 보고서에 작성

"+1" 에 유의할 것.
infix를 postfix로 바꿀 때, 바뀐 postfix 수식의 맨 끝에 END-OF_TOKEN ('\$') 추가하게 된다.
공간을 안전하게 확보하려면 +1 을 해야 한다.

□ Member Functions의 구현

■ Calculate 의 Public Member function의 구현

```

        else
            return false;
    }
    this->showOStackAll();
}
else { // currentToken이 일반 연산자 일때
    int inComingP = this->inComingPrecedence(currentToken);
    if (!this->_oStack.isEmpty()) {
        topToken = (char)this->_oStack.peek();
        while (this->inStackPrecedence(topToken) >= inComingP) {
            poppedToken = (char)this->_oStack.pop();
            this->_postfix[p++] = poppedToken;
            if (!this->_oStack.isEmpty())
                topToken = (char)this->_oStack.peek();
            else
                break;
        }
    }
    this->_oStack.push(currentToken);
    this->showOStackAll();
}
}
}
/* 적합한 과정을 추가하세요 */
return true;
}

```

- postfix 변환이 완료된 후, postfix 수식의 끝을 표시하는 문자 '\$' 를 추가한다.
- 함수의 알고리즘을 이해하고, 실제 입력 값을 예로 들어 소스에 대한 설명을 보고서에 작성

Precedence 처리



□ Precedence Table

- The operators with **Right-to-Left** Associativity:
In_coming precedence should be **higher** than In_stack precedence.
- The operators with **Left-to-Right** Associativity:
Both precedence values are the **same**.

Operator	In_stack precedence	In_coming precedence
(	0	20
)	19	19
+	12	12
-	12	12
*	13	13
/	13	13
%	13	13
^	16	17
\$	0	0

□ Precedence 를 위한 Private 함수

■ Calculate 의 Public Member function의 구현

```
private int inComingPrecedence(char aToken) {  
    if (aToken == '+')  
        return 12;  
    else if (aToken == '-')  
        return 12;  
    else if (aToken == '(')  
        return 20;  
    else if (aToken == ')')  
        return 19;  
    else if (aToken == '*')  
        return 13;  
    else if (aToken == '/')  
        return 13;  
    else if (aToken == '%')  
        return 13;  
    else if (aToken == '^')  
        return 17;  
    else if (aToken == '$')  
        return 0;  
    else  
        return -1;  
}
```

□ Precedence 를 위한 Private 함수

- Calculate 의 Public Member function의 구현

```
private int inStackPrecedence (char aToken) {  
    if ( aToken == '+' )  
        return 12;  
    else if .....  
        ..... // 적합한 연산자 우선 순위를 추가  
        // inComingPrecedence() 의 구현 코드를 참고  
}
```

Postfix 수식의 계산

□ 생각해볼 점 [1]

■ 전달받은 Token 을 double 값으로 바꾸는 방법

- Double.parseDouble (String.valueOf (token))

■ Postfix 수식에서:

- 문자 배열에 저장되어 있는 연산값(operand) token 들은 아직 정수 값으로 변환되지 않은 문자 상태이다.
- 연산을 위해서는 이 token 들을 정수 값으로 변환 (isDigit) 해야 한다.
- 그러므로 Postfix 수식을 저장한 문자 배열에서 operand 를 얻자마자 바로 정수 값으로 바꾸기로 한다.
- 정수로 바꾸고 해야 할 일은?

■ 계산할 때 사용하는 스택인 ValueStack 은?

- 스택에 들어가는 값의 자료형은 실수 (double) 이어야 한다.
 - ◆ 계산 결과가 반드시 정수 (int) 가 될 수 없다.
- 그러므로, ValueStack 의 원소의 generic type T 는 class "Double" (기본 자료형 "double" 이 아님) 로 한다.

□ 생각해볼 점 [2]

■ Postfix 수식에서:

- Token 이 연산자 일때 스택에 있는 값을 꺼내어 연산을 실행한다.
- 연산이 끝나면 계산이 끝난 값을 다시 스택에 넣는다.
- 연산이 끝나고 난 뒤, **스택의 top**에 남아있는 값이 **계산 결과 값**이다.
 - ◆ 이유는? postfix 수식을 계산하는 동안 일어나는 스택의 변환 과정을 살펴보세요.

Member Functions의 구현

Calculate 의 Public Member function의 구현

```
public double evalPostfix()
{
    int p = 0; // string pointer for postfix
    char curToken;
    this._vStack = new ArrayList<Double>(this._infix.length);

    curToken=this._postfix[p++];
    while(curToken != Calculate.END_OF_TOKEN) //Check for Token is END_OF_TOKEN '$'
    {
        if(this.isDigit(curToken)) {
            // Token is an operand : get value and push it into stack.
            /*
             * 적합한 과정을 추가하세요
             */
        }
        else {
            // Token is an operator.
            // Pop two operands, compute, and push the computed value.
            /*
             * 적합한 과정을 추가하세요
             */
        }
        curToken=this._postfix[p++];
    } //End of While (curToken == END_OF_TOKEN)
    //Now, the final result is on top of the stack.
    /* 적합한 과정을 추가하세요 */
    //pop and return this value
    return ...;
}
```

요약과 정리



□ 생각해 볼 점

- 보다 일반적인 계산기 프로그램을 작성하려면?
 - 한 자리 이상의 정수의 입력은?
 - 더 일반적으로, 실수 값의 입력은?
 - 부호가 붙은 수의 입력은?
- String 과 문자열 사이의 자료형의 변환
- 수식을 변환하고 계산하는 동안 일어나는 스택의 변환 과정을 살펴보시오.

보고서 작성



□ PDF 보고서 작성 방법

■ 겉장

- 제목: 자료구조 실습 보고서
- 부제목: [제08주] 수식 계산
- 제출일
- 학번/이름

■ 내용

1. 프로그램 설명서

1. 주요 알고리즘 /자료구조 /기타
2. 함수 설명서
3. 종합 설명서

2. 프로그램 장단점 분석

3. 실행 결과 분석

1. 입력과 출력 (화면 capture 시킬 것)
2. 결과 분석

4. 소스코드

□ 과제 제출

■ 파일 이름 작명 방법

- DS1_08_학번_이름.zip

- 폴더의 구성

- ◆ DS1_08_학번_이름

- 프로그램

- 프로젝트 폴더 / 소스

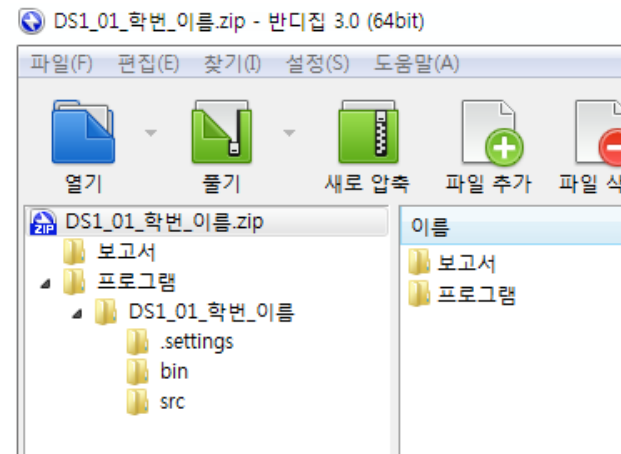
- 메인 클래스 이름 : DS1_08_학번_이름.java

- 보고서

- 이곳에 보고서 문서 파일을 저장한다.

- 입력과 실행 결과는 화면 image로 문서에 포함시킨다.

- 문서는 pdf 파일로 만들어 제출한다.



[제 8 주] 실습 끝



